

0.14. Deux choses, deux rythmes

Un compteur par chose : chacune avance à son rythme, et rien ne s'arrête.

1. Ce qu'il faut comprendre

Le problème : `attendre()` fige tout le programme. Impossible, avec lui, de faire changer une lettre toutes les secondes **et** une autre quatre fois par seconde : pendant que l'une attend, l'autre attend aussi.

La solution : on n'arrête plus jamais le programme. La boucle tourne sans cesse, un tour par image (60 par seconde), et **chaque chose a son propre compteur** d'images. C'est la méthode du 0.12, faite deux fois.

La lettre lente (ligne 0) : son compteur `imagesLente` avance d'un à chaque tour ; à **60**, une seconde est passée : elle change de lettre, et son compteur repart de 0.

La lettre rapide (ligne 2) : son compteur `imagesRapide` avance en même temps ; à **15**, un quart de seconde est passé ($60 \div 4 = 15$) : elle change, et son compteur repart de 0. Elle change donc **4 fois** pendant que la lente change **une fois**.

Les deux compteurs sont **indépendants** : on peut changer le 15 sans toucher au 60. Et comme la boucle ne s'arrête jamais, on pourrait lire la manette en même temps. C'est comme cela que tourne un jeu.

Le déroulé dans le temps : 0 s (tour 0) : A en haut, A en bas · **0,25 s** (tour 15) : `imagesRapide` atteint 15 et repart à 0, la rapide passe à **B** · **0,5 s** (tour 30) : la rapide passe à **C** · **0,75 s** (tour 45) : la rapide passe à **D** · **1 s** (tour 60) : `imagesLente` atteint 60 **et** `imagesRapide` atteint 15 : **les deux** changent, la lente passe à **B**, la rapide à **E**.

Rappels : `++` veut dire « ajoute 1 » ; `==` compare (« est égal à ? »), alors qu'un seul `=` range une valeur ; `sizeof(ALPHABET)` vaut 26, le nombre de lettres.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **940 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
démarrage      61 o ← la console, à l'allumage — la mise en route de la console
main()         91 o ← démarrage — ta fonction
VBlank         8 o  ← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage  28 o ← image() — attend l'image suivante
AttendreVBlank 16 o ← démarrage — attend que l'écran soit libre
CopierTuiles   18 o ← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage — vide les deux cartes de l'écran
Tuiles        704 o ← CopierTuiles — la police entière
Lettres gravées : toutes (poser() reçoit un numéro calculé)
```

Pas gravé (jamais appelé) : AvancerAirs, AttendreAcces, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireTexte, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

poser() × 2 24 o sur place, dans main()
image() × 1 3 o sur place, dans main() + AttendreImage (28 o)

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// _____  
// LES VARIABLES : écrites AVANT main(), elles existent pendant  
// tout le programme et gardent leur valeur d'un tour à l'autre.  
// uint8_t = un nombre entier de 0 à 255.  
// _____  
  
#include <ALPHABET>    // les lettres de la police : ALPHABET[0] est le A  
#include <poser>       // pose une tuile sur une case du fond
```

2. UNE VARIABLE — LENTE

```
uint8_t lente = 0;                // QUELLE lettre montre la lente : 0 = A, 1 = B ... 25 = Z
```

3. UNE VARIABLE — RAPIDE

```
uint8_t rapide = 0;               // QUELLE lettre montre la rapide : 0 = A, 1 = B ... 25 = Z
```

4. UNE VARIABLE — IMAGESLENTE

```
uint8_t imagesLente = 0;        // le CHRONOMÈTRE de la lente :  
                                 // combien d'images depuis son dernier changement
```

5. UNE VARIABLE — IMAGESRAPIDE

```
uint8_t imagesRapide = 0;       // le CHRONOMÈTRE de la rapide, séparé de l'autre
```

6. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {

    // La boucle de jeu : elle ne s'arrête JAMAIS.
    // Chaque tour dure exactement une image, soit 1/60e de seconde.
    while (true) {

        image();                // attend l'image suivante (≈ 16,7 ms) :
                                // c'est ce qui cadence la boucle à 60 tours par seconde

        // À chaque tour, UNE image est passée : les deux chronomètres
        // avancent d'un cran, en même temps, chacun de son côté.
        imagesLente++;          // ++ veut dire « ajoute 1 »
        imagesRapide++;

        // — LA LENTE : elle change toutes les 60 images = toutes les secondes —
        if (imagesLente == 60) { // == veut dire « est égal à ? »
                                // (un seul = voudrait dire « range »)
            imagesLente = 0;      // son chronomètre repart de zéro
            lente++;              // elle passe à la lettre suivante
            if (lente == sizeof(ALPHABET)) lente = 0; // sizeof(ALPHABET) = 26 :
                                // après Z (25), 26 n'existe pas → retour à A (0)
        }

        // — LA RAPIDE : elle change toutes les 15 images = 4 fois par seconde —
        // (60 images ÷ 15 = 4 changements par seconde)
        if (imagesRapide == 15) {
            imagesRapide = 0;      // son chronomètre repart de zéro
            rapide++;              // elle passe à la lettre suivante
            if (rapide == sizeof(ALPHABET)) rapide = 0; // après Z, retour à A
        }

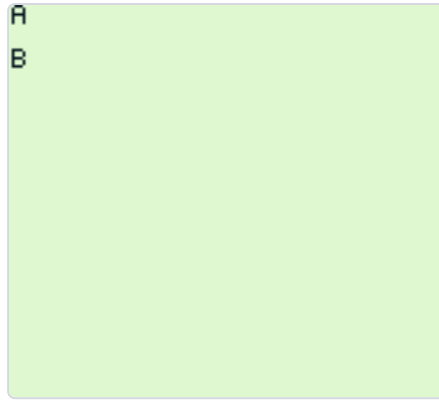
        // — L'AFFICHAGE : à CHAQUE tour, on repose les deux lettres —
        // Si rien n'a changé, on repose la même lettre au même endroit : ça ne se voit pas.
        // Si l'indice a changé, la nouvelle lettre écrase l'ancienne (comme au 0.3).
        poser(0, 0, ALPHABET[lente]); // colonne 0, ligne 0 : la lente
        poser(0, 2, ALPHABET[rapide]); // colonne 0, ligne 2 : la rapide
                                        // (la ligne 1 reste vide pour les séparer)
    }
}
```

3. Ce qui se passe, étape par étape

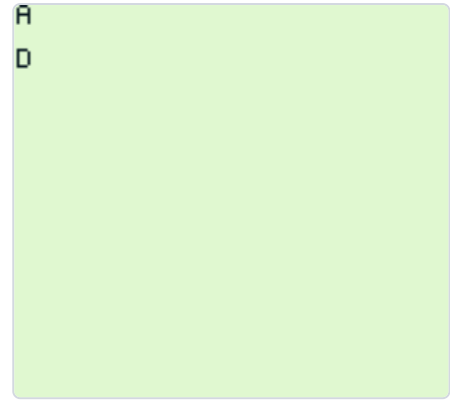
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



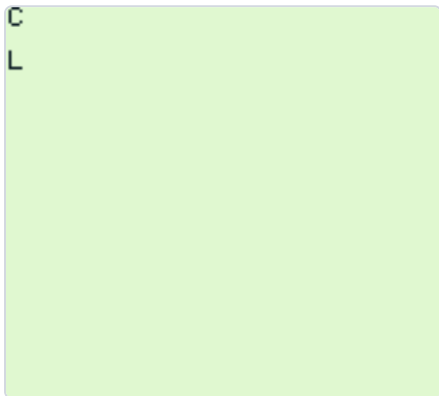
la toute première image — la console vient de démarrer — image 1



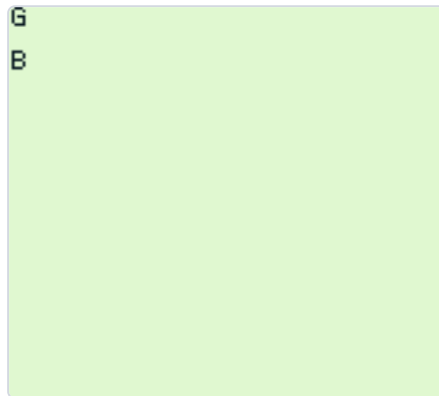
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

Deux lettres : celle du bas change quatre fois pendant que celle du haut change une fois.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ la rapide change à chaque quart de seconde (8 fois en 2 s) (AB AC AD BE BF BG BH CI CJ)
- ✓ la lente change toutes les secondes (2 fois en 2 s)

Ensuite : **0.14.1. Deux choses, deux rythmes — de base, ailleurs** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++